

Chapter 6

Modern Asynchronous I/O — io_uring

5 Lecture · 영상 87 분 · Ubuntu Server 26.04 + C
Observe → Understand → Implement → Analyze

L26 Linux Async I/O 는 왜 필요해졌는가

L27 io_uring Architecture

L28 SQ / CQ / SQE / CQE 내부 구조

L29 io_uring File Read / Write 실습

L30 io_uring 심화와 기존 I/O 비교

L26

Linux Async I/O 는 왜 필요해졌는가

Chapter 6. Modern Asynchronous I/O — io_uring

Disk 의 File 은 언제나 " 읽을 수 있다 " 고 답하고는 , 읽으면 잠든다

영상 16 분

▶ 애니메이션 [C6-1 Readiness vs Completion](#)

문제 상황

```
[ 30.1 ms] tick 3 ( 만료 횟수 1)
           Handler: 256MB read 에 1207.4 ms
[ 1237.6 ms] tick 4 ( 만료 횟수 121) ← 그동안 Loop 가 멈춰 있었다
```

10ms 심장 박동의 Event Loop. Handler 에서 File 을 읽었더니 **1.2 초 동안 전체가 멈춤**

O_NONBLOCK 으로 열었지만 효과 없음

→ *Terminal 전환*

일반 File 은 언제나 " 준비됨 "

```
epoll_ctl(ADD, 일반 File) = -1, EPERM  
poll(일반 File)           = 1, revents = POLLIN | POLLOUT ← 항상
```

ext4 의 file_operations 에는 .poll 이 없다

File 의 " 준비됨 " = " 읽으려는 그 **Offset** 의 **Page** 가 Cache 에 있다 " → read 를 해 봐야 안다

- **Readiness** 라는 질문 자체가 File 에는 성립하지 않는다

Readiness 와 Completion

	Readiness	Completion
질문	지금 하면 Block 되지 않는가 ?	부탁한 일이 끝났는가 ?
순서	통지 → 내가 read	요청 → Kernel 이 읽음 → 통지
대표	select, poll, epoll	io_uring, IOCP, POSIX AIO
Disk File	맞지 않는다	잘 맞는다
Buffer 를 넘기는 시점	읽기 직전	요청할 때 (완료까지 유지)

Readiness vs Completion

화면 전환 → [animation.html](#)

epoll + pipe → epoll 식으로 File 을 읽으면 → Completion

▶ 애니메이션 [C6-1 Readiness vs Completion](#)

기존 해법 1: Thread Pool, POSIX AIO

`aio_read` 호출 전 Thread 수 : 1

`aio_read` 4 개 요청 직후 Thread 수 : 2 ← `glibc` 가 Worker Thread 를 만들었다

`glibc` 의 `aio_read` = 숨은 Thread 가 Blocking `pread` 를 대신 호출 . Kernel 은 모른다

- 요청마다 Thread 간 전달 : Queue + Lock + `futex` + Context Switch × 2
- 동시 I/O 수 = Thread 수 . NVMe 는 수만 개를 동시에 받는다
- HDD(10ms) 에서는 묻히던 비용이 NVMe(수십 μ s) 에서 드러났다

→ Terminal 전환

기존 해법 2: Linux Native AIO

`io_setup / io_submit / io_getevents` (libaio). Thread 없는 진짜 비동기. 그러나

O_DIRECT 전용. Buffered에서는 `io_submit`이 조용히 Block

- File read/write 만. open, stat, Socket 미지원
- 요청마다 System Call 과 복사

→ 자체 Cache 를 가진 Database 만의 도구 (L14)

또 하나의 압력 : System Call 비용

장치	I/O 1 회	System Call 비용의 비중
HDD	10,000 μ s	무시
SATA SSD	100 μ s	1%
NVMe	10~20 μ s	5~10%
Page Cache Hit, Optane	1~10 μ s	수십 %

+ Spectre/Meltdown 대응으로 전환 비용 증가 (2018)

"I/O 한 번에 System Call 한 번 (epoll 은 두 번)" 이라는 구조 자체가 병목

io_uring 의 설계 목표 (Linux 5.1, 2019)

목표	방법
Buffered 를 포함한 진짜 비동기 File I/O	가능하면 즉시 , 아니면 Kernel 내부에서 비동기로
System Call 줄이기	공유 Memory 의 Ring Buffer. Batch, 설정에 따라 0 번
요청마다의 복사 없애기	Ring 을 mmap 으로 공유
범용 Interface	read/write, open, fsync, statx, timeout, poll, Socket ... 60 여 종

epoll 을 대체하는 것이 아니다 . 목적이 다르고 함께 쓸 수 있다

정리

- Readiness(epoll) vs Completion
- Disk File 은 Readiness 에 맞지 않는다 → Event Loop 가 멈춘다
- Thread Pool: 전달 비용 / Native AIO: O_DIRECT 전용
- System Call 자체가 병목이 되었다
- io_uring: 진짜 비동기 + 공유 Ring + 범용

다음 : L27. 공유 Memory 의 Ring Buffer 란 실제로 어떤 구조인가

L27

io_uring Architecture

Application 과 Kernel 이 Memory 를 공유하는 두 개의 Ring Buffer

영상 17 분

▶ 애니메이션 [C6-2 io_uring Architecture](#)

준비 : liburing

```
sudo apt install -y liburing-dev          # Link: -luring
```

- io_uring 의 System Call 은 glibc 에 Wrapper 가 없다
- Ring 을 직접 다루는 것은 번거롭고 실수하기 쉽다 (L28)
- liburing = io_uring 제작자가 관리하는 얇은 C Library

전체 구조

Application 공유 Memory (mmap) Kernel
 SQE 작성, tail 전진 → Submission Queue (SQ) → 꺼내서 실행, head 전진
 CQE 읽기, head 전진 ← Completion Queue (CQ) ← 결과 작성, tail 전진

`io_uring_enter()` ← "SQ 에 넣었으니 가져가라 " / " 완료될 때까지 재워 달라 "

SQE	요청 하나 , 64Byte: opcode, fd, addr, len, off, user_data
CQE	결과 하나 , 16Byte: user_data , res, flags

세 개의 System Call

System Call	역할	빈도
io_uring_setup	Ring 생성 , fd return	한 번
io_uring_enter	제출을 알리고 , 원하면 완료 대기	여러 요청에 한 번
io_uring_register	Buffer, File, eventfd 사전 등록	가끔

가장 작은 io_uring 프로그램

```
io_uring_queue_init(8, &ring, 0);  
sqe = io_uring_get_sqe(&ring);  
io_uring_prep_read(sqe, fd, buf, n, 0);  
io_uring_sqe_set_data64(sqe, 42);  
io_uring_submit(&ring);  
io_uring_wait_cqe(&ring, &cqe);  
printf("%llu %d\n", cqe->user_data, cqe->res);  
io_uring_cqe_seen(&ring, cqe);
```

/* System Call 아님 */

/* io_uring_enter */

/* 42, 읽은 Byte 수 */

오류는 **errno** 가 아니라 **-errno return 값**과 **CQE 의 res** 로

→ *Code + Terminal*

strace 로 본 System Call

```
io_uring_setup(8, {...sq_entries=8, cq_entries=16...}) = 4
mmap(NULL, 464, ..., MAP_SHARED|MAP_POPULATE, 4, 0)      = 0x7f...
mmap(NULL, 512, ..., MAP_SHARED|MAP_POPULATE, 4, 0x100000000) = 0x7f...
io_uring_enter(4, 1, 0, 0, NULL, 8)                      = 1
io_uring_enter(4, 0, 1, IORING_ENTER_GETEVENTS, NULL, 8) = 0
```

read 가 없다 . File 을 읽었는데 read System Call 은 호출되지 않았다

→ Terminal 전환

요청 하나의 일생

화면 전환 → [animation.html](#)

Setup → mmap → SQE 작성 → enter → Kernel 처리 → CQE → 소비

▶ 애니메이션 [C6-2 io_uring Architecture](#)

liburing 없이 : mmap 된 Ring

```
7f24...04000-...05000 rw-s 100000000 ... anon_inode:[io_uring]      ← SQE 배열
7f24...05000-...06000 rw-s 000000000 ... anon_inode:[io_uring]      ← SQ/CQ Ring
3. SQE[0] 작성, SQ tail 0 → 1 (System Call 없음)
4. io_uring_enter(to_submit=1, min_complete=1) = 1
5. CQE: user_data=0xCAFE res=9
```

`rw-s` = `MAP_SHARED` (L10) · `anon_inode` (L25)

SQE = System Call 의 인자를 **Register** 대신 공유 **Memory** 에 적은 것

→ *Code + Terminal*

Kernel 안에서의 처리 , 그리고 환경

1. Inline 시도 (NOWAIT): Page Cache Hit 면 그 자리에서 완료
2. Block 될 상황이면
 - Pipe/Socket: 내부 Poll 후 실행 (FAST_POLL)
 - Buffered File: Page 가 올라오면 이어서
 - O_DIRECT: Block Layer 에 비동기 제출
 - 그 외: Kernel Worker (io-wq, iou-wrk-<pid>)

쓸 수 없는 환경이 많다: kernel.io_uring_disabled, Docker 기본 seccomp, Android, ChromeOS

→ EPERM / ENOSYS 에 대비해 대체 경로 준비

→ Terminal 전환: ./uring_probe

정리

- mmap 으로 공유하는 Ring 두 개 : SQ(요청), CQ(결과)
setup / **enter** / register. SQE 쓰기와 CQE 읽기는 Memory 접근
liburing. 오류는 -errno
- Inline → Poll / 비동기 Block I/O / io-wq
- 환경 확인

다음 : L28. head 와 tail, Lock 없이 공유하는 Queue

L28

SQ / CQ / SQE / CQE 내부 구조

head 와 tail 숫자 두 개 , 그리고 Memory Barrier. Lock 없이 Kernel 과 공유하는 Queue

영상 17 분

▶ 애니메이션 [C6-3 SQ / CQ Ring Buffer](#)

Ring Buffer: head, tail, mask

```

entries = 4, mask = 3          [0]    [1]    [2]    [3]
                                req B  req C
                                ↑ head = 5 (5 & 3 = 1)    ↑ tail = 7 (7 & 3 = 3)
개수 = tail - head = 2        빈 상태 : head == tail        가득 : tail - head == entries
  
```

entries 는 **2** 의 거듭제곱 → 위치 = $\text{index} \& \text{mask}$

head, tail 은 계속 증가 (감기지 않는다)

Producer 는 tail 만 , **Consumer** 는 head 만 쓴다

	Producer	Consumer
SQ	Application	Kernel
CQ	Kernel	Application

움직임을 직접 본다

0. 초기	SQ head=0 tail=0 (local tail=0) CQ head=0 tail=0	
1. <code>get_sqe</code> × 3	SQ head=0 tail=0 (local tail=3) CQ head=0 tail=0	← 아직 비공개
2. <code>submit</code> 후	SQ head=3 tail=3 CQ head=0 tail=3	
3. CQE 3 개 소비 후	SQ head=3 tail=3 CQ head=3 tail=3	
4. 3 개 더	SQ head=6 tail=6 CQ head=3 tail=6	← 6 & 3 = 2 번 칸

`get_sqe` 는 local tail 만 . **submit** 이 **tail** 을 한 번에 공개 → Batch

→ Terminal 전환

왜 Lock 이 필요 없는가

SQ: Application - 쓰다 → tail, SQE

Kernel - 쓰다 → head

각 변수를 쓰는 쪽이 **하나뿐** = SPSC Queue. CAS 재시도도 ABA 도 없다 (L20)

필요한 것은 순서 하나 :

```
sqes[idx] = ...; /* 1. 내용을 먼저 쓰고 */
io_uring_smp_store_release(sq->ktail, t); /* 2. tail 을 공개: Memory Barrier (L18) */
```

하나의 Ring 을 여러 Thread 가 쓰면 전제가 깨진다 → **Thread** 마다 **Ring** 하나

Ring Buffer 직접 조작하기

화면 전환 → [animation.html](#)

Scenario 3 개 + Sandbox: Create SQE / Submit / Kernel Process / Complete / Read CQE

▶ 애니메이션 [C6-3 SQ / CQ Ring Buffer](#)

SQE: 64Byte 에 담긴 System Call

```
struct io_uring_sqe { opcode; flags; fd; off; addr; len; ...; user_data; ... };    /* 64 Byte */
```

liburing	System Call
prep_read(sqe, fd, buf, n, off)	pread
prep_fsync	fsync
prep_openat	openat
prep_timeout	(timerfd)
prep_poll_add	(epoll)

Offset 은 항상 명시 (공유 f_pos 에 의존하지 않는다)

- 가리키는 **Memory** 는 완료까지 살아 있어야 한다

CQE 와 user_data

```
struct io_uring_cqe { __u64 user_data; __s32 res; __u32 flags; };    /* 16 Byte */
```

CQE 에는 fd 도 opcode 도 **없다** . 요청과 완료를 잇는 것은 user_data 뿐 (= epoll 의 data)

실전 : io_uring_sqe_set_data(sqe, req) → 요청 구조체의 포인터

완료는 순서대로 오지 않는다

```
제출 : A (300ms)    B (100ms)    C (200ms)
[100.1 ms] 완료 : B    res=-62 (Timer expired)
[200.2 ms] 완료 : C
[300.1 ms] 완료 : A
```

제출 A, B, C → 완료 **B, C, A**

- 동시에 진행 : 전체 300ms (600ms 아님)

res 의 의미는 opcode 마다 다르다 (TIMEOUT 의 -ETIME 은 정상 만료)

→ *Terminal 전환*

오류 , SQ Full, CQ Overflow

`submit = 2` ← 제출 자체는 성공

요청 1: `res = -9` → Bad file descriptor

요청 2: `res = -21` → Is a directory

`get_sqe` 가 NULL 을 return 하기 전까지 : 4 개

	대응
연산 실패	모든 CQE 의 <code>res</code> 확인
SQ Full	<code>get_sqe = NULL</code> → <code>submit</code> 후 재시도
CQ Overflow	CQE 를 부지런히 거둔다 . In-flight 수 제한
Short Read/Write	남은 부분을 새 SQE 로

→ Terminal 전환

정리

head/tail/mask. 위치 = & mask, 개수 = tail - head

- SPSC → Lock 불필요. 순서 + Memory Barrier
- get_sqe 는 local, submit 이 공개 → Batch
- SQE 64B = System Call 인자. Offset 명시, Memory 수명
- CQE 16B = user_data + res
- 완료는 순서대로 오지 않는다. 오류는 res 로

다음 : L29. 실제로 File 을 읽고 쓴다

L29

io_uring File Read / Write 실습

나누고, 묶어서 제출하고, 뒤섞여 돌아오는 완료를 조립한다. 그리고 정직하게 측정한다

영상 18 분

▶ 애니메이션 [C6-4 Batch와 System Call](#)

기본 Loop 의 모양

```
while ( 할 일이 남았다 ) {
    ① 채우기      : get_sqe → prep → set_data          (SQ 에 자리가 있는 동안 )
    ② 제출 + 대기 : io_uring_submit_and_wait(&ring, 1) ← System Call 은 여기 한 번
    ③ 거두기      : peek_cqe → res 확인 → cqe_seen      (System Call 없음 )
}
```

	System Call
submit_and_wait(ring, k)	enter 1 회 (제출 + k 개 대기)
peek_cqe, cqe_seen	없음

uring_cat: 설계 Point

```
io_uring_prep_read(sqe, fd, data + off, len, off);    /* 목적지 = 최종 위치 */
io_uring_sqe_set_data(sqe, &reqs[i]);                /* 요청 구조체 */
...
while (submitted < nblocks && inflight < QD) { ... } /* In-flight 제한 */
io_uring_submit_and_wait(&ring, 1);                  /* 하나라도 끝나면 돌아와 다시 채운다 */
```

각 요청이 최종 위치에 직접 읽는다 → **완료 순서에 의존하지 않는다**

`inflight < QD` → CQ 가 넘치지 않는다

`wait 1` → Pipeline 이 계속 차 있다

→ *Code 화면*

실행과 검증

```
$ ./uring_cat /usr/bin/gcc | cmp - /usr/bin/gcc && echo same  
[uring_cat] 1023032 Byte, read 요청 250 개, io_uring_enter 4 회  
same
```

read 250 번 분량을 **System Call 4 번**으로 . 원본과 Byte 단위로 동일

→ *Terminal 전환*: `strace -c`

Batch 의 효과 측정

64MB 를 4KB 씩 16384 번 읽기 (모두 Page Cache Hit)

pread() Loop	16384 syscalls	9.7 ms
--------------	----------------	--------

io_uring batch 1	16384 syscalls	12.2 ms	← 더 느리다
------------------	----------------	---------	---------

io_uring batch 8	2048 syscalls	8.2 ms
------------------	---------------	--------

io_uring batch 64	256 syscalls	7.5 ms
-------------------	--------------	--------

io_uring batch 256	64 syscalls	7.7 ms
--------------------	-------------	--------

System Call 은 **1/256**, 시간은 **-23%**

→ Terminal 전환

정직하게 해석하기

비용	pread	io_uring batch 64
User ↔ Kernel 전환	16,384	256 ← 줄어든 것은 이것뿐
VFS, Page Cache 조회	16,384	16,384
64MB 복사	같다	같다 ← 시간의 대부분
SQE/CQE 처리	없음	16,384 (새 비용)

io_uring 의 이득은 두 종류 :

System Call 절감 : " 전환 비용 / 요청당 실제 작업 " 이 클수록

기다리지 않음 : Disk 를 실제로 기다리는 I/O, O_DIRECT, fsync → **대개 이쪽이 훨씬 크다 (L30)**

Batch 와 System Call

화면 전환 → [animation.html](#)

`read() × 8 → io_uring Batch → SQPOLL`

▶ 애니메이션 [C6-4 Batch와 System Call](#)

Linked SQE: read → write → fsync

```
io_uring_prep_read(sqe, in, buf, len, off);    sqe->flags |= IOSQE_IO_LINK;
io_uring_prep_write(sqe2, out, buf, len, off);    /* read 가 성공한 뒤에 실행 */
...
io_uring_prep_fsync(sqe, out, 0);                /* fsync 도 요청 하나 (L15) */
```

제출된 요청들은 **병렬**로 실행된다 . 순서가 필요하면 Link
앞이 실패 (Short Read 포함) 하면 뒤는 -ECANCELED

- fsync: 수 ms~ 수십 ms Block 되는 대표적 연산 → 비동기의 이득이 크다

→ Code + Terminal: [uring_cp](#)

실전 Checklist

모든 CQE 의 res 확인 / Short I/O 재제출
get_sqe 의 NULL / In-flight 제한

Buffer 수명 : 완료까지 해제 · 재사용 금지

- Offset 명시
- 종료 전에 진행 중인 요청을 모두 거둔다

EPERM/ENOSYS 대체 경로

- Ring 하나는 한 Thread 에서

정리

- 채우기 → submit_and_wait → 거두기
 - 요청 구조체 + user_data, 최종 위치에 직접, In-flight 제한
 - Batch: System Call 1/256, 시간 -23%. Batch 1 은 더 느리다
- 이득 = System Call 절감 + **기다리지 않음**
- IOSQE_IO_LINK, 비동기 fsync

다음 : L30. System Call 0 번 , 사전 등록 , Event Loop 통합 , 그리고 비교

L30

io_uring 심화와 기존 I/O 비교

Event Loop 통합 , System Call 0 번 , 사전 등록 . 그리고 다섯 가지 I/O 모델

영상 19 분

▶ 애니메이션 [C6-5 I/O Model 비교](#)

Event Loop 와 합치기

```
efd = eventfd(0, EFD_NONBLOCK);  
io_uring_register_eventfd(&ring, efd);      /* CQE 가 생길 때마다 Kernel 이 efd 에 write */  
epoll_ctl(epfd, EPOLL_CTL_ADD, efd, &ev);  /* 나머지는 Chapter 5 와 같다 */
```

L25 의 답 그대로 : **fd** 로 만들면 **epoll** 에 넣을 수 있다

Handler: read() 대신 요청 256 개를 제출만 하고 **Loop** 로 복귀

L26 의 문제가 해결되었는가

Mode: 0_DIRECT

[30.1 ms] tick 3

Handler: read 요청 256 개 제출에 3.05 ms → Loop 로 복귀

[1262.6 ms] 256MB 읽기 완료 . 그동안 tick 은 126 까지 진행

놓친 Tick: 0

	loop_stall (L26)	uring_eventfd direct
Handler 점유	1.2 초	3ms
놓친 Tick	약 120	0

Thread 를 만들지 않았다 . Signal, Timer, Worker 통지 , **File I/O** 가 하나의 `epoll_wait` 에서

→ *Terminal 전환*

Buffered 에서는 ?

Mode: Buffered

놓친 Tick: 3

(환경에 따라 0 ~ 수 개)

Buffered : Disk →DMA→ Page Cache →CPU 복사→ User Buffer

↑ 이 대기는 숨겨진다

↑ 이 복사는 누군가의 CPU 시간

O_DIRECT : Disk →DMA→ User Buffer

(CPU 가 할 일이 없다)

비동기 I/O 가 없애는 것은 " 기다림 " 이지 " 일 " 이 아니다

O_DIRECT + io_uring = 복사도 , 대기도 , Thread 도 없다 → Database 가 쓰는 방식 (L14)

→ Terminal 전환

SQPOLL: System Call 0 번

```
p.flags = IORING_SETUP_SQPOLL;    p.sq_thread_idle = 2000;
```

375 377 iou-sqp-375 ← SQ 를 계속 들여다보는 Kernel Thread
read 20000 회 완료 . io_uring_enter: 1 회

대가	
Core 하나를 Polling 에	L21 의 Busy Polling 을 Kernel 이 대신
Idle 후에는 Wakeup 필요	liburing 이 자동 처리
완료를 " 기다리려면 " 여전히 System Call	

Latency 를 위해 CPU 를 지불한다 (L20 의 Spin vs Sleep)

→ Terminal 전환: *ps, strace -c*

사전 등록

매 요청마다	등록해 두면
fd → struct file 조회 , 참조 Count ±	register_files + IOSQE_FIXED_FILE (fd 자리에 Index)
(O_DIRECT) Buffer Page 고정 / 해제	register_buffers + prep_read_fixed

일반 READ	28.4 ms (81,920 회)	
READ_FIXED + IOSQE_FIXED_FILE	24.3 ms	← 요청당 약 50ns

요청당 작업이 작을수록 의미가 있다 . ulimit -l 확인

→ Terminal 전환

그 밖의 기능

POLL_ADD: Readiness 대기 = epoll 이 하던 일

Multishot: 요청 하나로 완료 여러 번

- TIMEOUT, LINK_TIMEOUT, ASYNC_CANCEL
- Provided Buffers, IOPOLL
- Network: accept, send, recv, Zero Copy (Network 과정에서)

→ epoll 없이 io_uring 하나로 Event Loop 구성도 가능

다섯 가지 I/O 모델

화면 전환 → [animation.html](#)

Blocking → Non-blocking → epoll → POSIX AIO → io_uring

▶ 애니메이션 [C6-5 I/O Model 비교](#)

비교

	Blocking	epoll	POSIX AIO	io_uring
I/O 수행 주체	호출 Thread	호출 Thread	숨은 Thread	Kernel
I/O 당 System Call	1	2	1 + 전달	1/N, 0
Disk File	Block	불가	가능	가능
Pipe/Socket/Timer	0	주 용도	부적합	0
복잡도	최저	중	중	최고
이식성	POSIX	Linux	POSIX	5.1+, 차단 많음

" 어느 것이 가장 빠르냐 " 는 의미 있는 질문이 아니다 . 풀려는 문제가 다르다

선택 기준

상황	선택
File 몇 개를 순서대로	Blocking
많은 Pipe/Socket/Timer/Signal	epoll
epoll Loop 에 File I/O 가 조금	Worker + eventfd, 또는 io_uring + eventfd
Disk I/O 가 핵심 , O_DIRECT, 잦은 fsync	io_uring
초당 수십만 I/O, Latency 최우선	+ SQPOLL, Registered
Container, Android	대체 경로 필수

Chapter 6 정리

문제	: Disk File 은 Readiness 에 맞지 않는다 . Thread Pool 의 비용 , System Call 의 비용	L26
구조	: mmap 으로 공유하는 Ring 두 개 . SQE(64B) → CQE(16B) . setup / enter / register	L27
내부	: head·tail·mask, SPSC + Memory Barrier, user_data, 순서 없음 , 오류는 res	L28
사용	: 채우기 → submit_and_wait → 거두기 . Batch 의 이득은 제한적 . LINK, 비동기 fsync	L29
심화	: eventfd 통합 , O_DIRECT + io_uring, SQPOLL, Registered	L30

Worksheet 20 문제 → Chapter 7: Debugging, Tracing, Performance

다음 시간 · Chapter 7

Linux Debugging, Tracing, Performance

개념 워크시트

[worksheet.html](#)

Lecture 당 10 문항 · 50 문항

애니메이션

[animation.html](#)

5 종

강의 자료

[lecture.html](#)

빈칸 채우기 · 괄호 넣기

저장소 github.com/imguru-mooc/adv-sysprog · 온라인 자료 imguru-mooc.github.io/adv-sysprog